

Guidelines for Functions

When writing production-level code, you have to assume that whoever reads your code in the future is tired, rushed, and has zero context about what you were trying to do. Your functions should be **"idiot-proof."**

Guideline 1: Self-Documenting Names

The name of your function and its parameters should instantly explain what the code does without forcing the reader to look at the actual logic inside.

- **Bad:** `func process(a []float64)` What are we processing? What is a?
- **Good:** `func ComputeRMS(samples []float64)` Instantly clear: we are computing the Root Mean Square using a slice of audio or data samples.

Note: Keep names domain-specific but avoid making them ridiculously long (e.g., `func ComputeRootMeanSquareOfAudioSamplesFromDatabase()`).

Guideline 2: Functional Cohesion (The "Single Responsibility Principle")

A function should do **exactly one thing**, and its name should reflect that one thing.

- **The Trap:** Sometimes, developers group operations together just because they happen at the same time in the program.

Example: `func DrawCircleAndComputeTriangleArea()`

- **Why it's bad:** It confuses the human brain, makes the function impossible to name cleanly, and makes debugging a nightmare. If the triangle math breaks, the developer now has to dig through circle-drawing code to find the bug.
- **The Fix:** Split them up! `DrawCircle()` and `ComputeTriangleArea()`.

Guideline 3: Reduce the Number of Parameters

If a function takes **10** or **20** parameters, it is a massive red flag.

1. It usually means you violated Guideline 2, the function is trying to do too many things at once.
2. It makes tracing bugs incredibly difficult because you have to check 20 different variables to see which one is corrupted.

How to fix parameter bloat: Group related data into an struct (object)

- **Bad (9 Parameters):** `func CalcTriangleArea(x1, y1, z1, x2, y2, z2, x3, y3, z3 float64)`
- **Better (3 Parameters):** `func CalcTriangleArea(p1 Point, p2 Point, p3 Point)`
- **Best (1 Parameter):** `func CalcTriangleArea(t Triangle)`

Expert Takeaway:

These three guidelines perfectly describe what we call "**Clean Code**." When you are building APIs in Go, this concept of grouping data is incredibly important.

If an endpoint registers a new user, you don't pass **firstName**, **lastName**, **email**, **password**, and **age** into your database function. You define a **User struct**, bind the incoming **JSON** to that single struct, and then pass that one User object down the chain. It keeps your Go backend lean, readable, and highly professional!